

KONGU ENGINEERING COLLEGE
(AUTONOMOUS)

PERUNDURAI, ERODE – 638 060

EXPERIMENT - 9

PROJECT TITLE

**INDUSTRIAL DIESEL GENERATOR MONITORING
AND MAINTENANCE SYSTEM**

SUBMITTED BY

ABISHEK R S - 24MER002

GOWRISANKAR K - 24MER010

NAVEENRAJ K - 24MER035

PRAVEEN N M - 24MER043

INDUSTRIAL DIESEL GENERATOR MONITORING AND MAINTENANCE SYSTEM

1. Project review:

The Industrial Diesel Generator Monitoring and Maintenance System is a compact prototype designed to continuously monitor important operating parameters of a diesel generator and provide an early warning when abnormal conditions occur.

The system uses an ESP8266 NodeMCU as the central controller along with sensors to monitor parameters such as temperature and vibration. The sensor readings are continuously processed and compared with predefined safety limits.

When abnormal temperature or excessive vibration is detected, the system activates a buzzer and LED to provide a warning. A relay module can also be used to demonstrate generator protection or automatic shutdown.

The proposed system provides a simple, low-cost, and automatic method for industrial generator condition monitoring and maintenance. It can be further developed using IoT connectivity for remote monitoring, cloud data storage, maintenance alerts, and historical performance analysis.

2. Problem Statement:

Industrial diesel generators are important backup and power-generation systems. During operation, generators may experience problems such as overheating, excessive vibration, and abnormal operating conditions.

If these problems are not detected at an early stage, they may lead to generator failure, increased maintenance cost, equipment damage, and interruption of power supply.

Traditional maintenance methods may depend on manual inspection and periodic maintenance. Therefore, there is a need for an automatic system that can continuously monitor generator conditions and provide an early warning when abnormal conditions occur.

The proposed system uses sensors, ESP8266 NodeMCU, buzzer, LED, and relay module to monitor the generator condition and provide maintenance alerts

3. Proposed Solution:

The proposed solution consists of an ESP8266-based Industrial Diesel Generator Monitoring and Maintenance System.

A temperature sensor is used to monitor the generator operating temperature, while a vibration sensor is used to detect abnormal vibration. The sensor readings are sent to the ESP8266 NodeMCU.

The ESP8266 processes the sensor values and compares them with predefined limits. If excessive temperature or abnormal vibration is detected, the system activates the buzzer and LED. The relay can also be activated to demonstrate generator protection or shutdown.

Proposed Operation:

1. Start the system and initialize the ESP8266.
2. Initialize the temperature sensor, vibration sensor, relay, buzzer, and LED.
3. Continuously monitor generator temperature.
4. Continuously monitor generator vibration.
5. Send the sensor readings to the ESP8266.
6. Compare the readings with predefined safety limits.
7. If the generator condition is normal, continue monitoring.
8. If high temperature or abnormal vibration is detected, activate the warning system.
9. Turn ON the buzzer and LED.
10. Activate the relay for protection or shutdown.
11. Display a maintenance warning.
12. Repeat the monitoring process continuously.

4. System Architecture:

The Industrial Diesel Generator Monitoring and Maintenance System consists of five main sections:

Input Section:

Temperature Sensor – Measures the operating temperature of the generator.

Vibration Sensor – Detects excessive or abnormal generator vibration.

The sensors send the measured data to the ESP8266 NodeMCU.

Processing Section:

The ESP8266 NodeMCU acts as the central controller.

- Receives sensor data.
- Processes the temperature and vibration readings.
- Compares the values with predefined limits.
- Detects abnormal generator conditions.
- Controls the relay, buzzer, and LED.

Actuation Section:

The Relay Module is used for generator protection.

- The relay receives a control signal from the ESP8266.
- It can demonstrate automatic generator shutdown.
- It can also be connected to an external protection or warning device.

Indication Section:

The Buzzer and LED provide warning indications.

- Buzzer OFF and LED OFF → Generator condition normal.
- Buzzer ON and LED ON → Generator abnormal condition detected.

Connectivity Section:

The ESP8266 has built-in Wi-Fi capability.

The system can be further developed for:

- IoT-based generator monitoring.
- Remote condition monitoring.
- Cloud data storage.
- Mobile maintenance alerts.
- Generator performance history.

- Predictive maintenance analysis.
- Energy management.

5. Circuit Table:

Important Pin Mapping:

ESP8266 NodeMCU Pin Component

A0	A0 Temperature Sensor → Analog Output
GPIO 2	D4 Vibration Sensor → OUT
GPIO 14	D5 Relay Module → IN
GPIO 12	D6 Buzzer → Positive Terminal
GPIO 13	D7 LED → Anode through resistor
3.3V / 5V	– Sensor Power
GND	– Common Ground

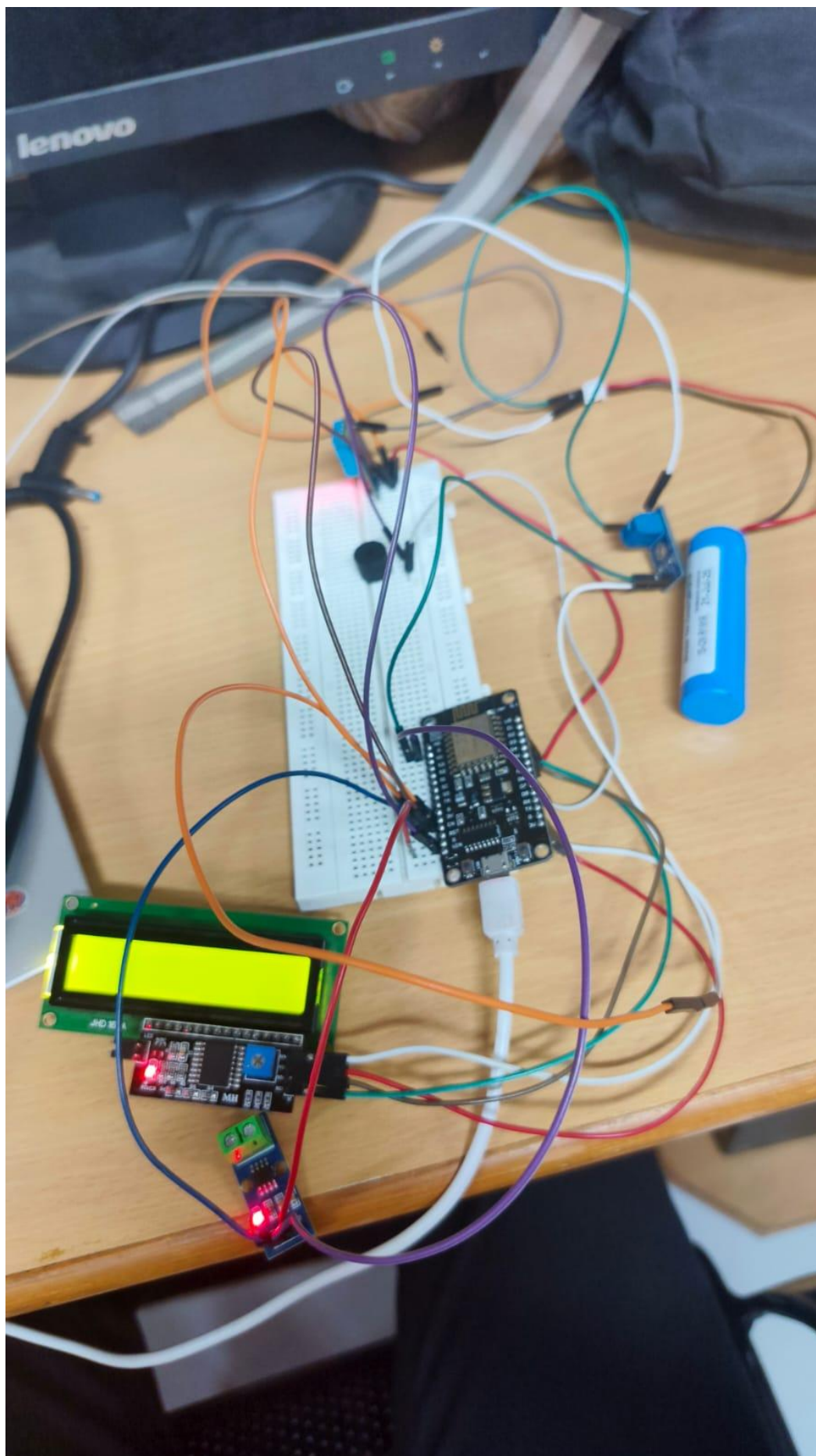
6. Circuit Design:

The circuit consists of an ESP8266 NodeMCU, temperature sensor, vibration sensor, relay module, buzzer, LED, breadboard, and jumper wires.

The temperature sensor is connected to the A0 analog input of the ESP8266 to monitor the generator temperature. The vibration sensor output is connected to GPIO2 (D4) to detect abnormal vibration.

The relay module is connected to GPIO14 (D5) and can be used for generator protection or automatic shutdown. The buzzer is connected to GPIO12 (D6) and provides an audible warning. The LED is connected to GPIO13 (D7) through a suitable resistor and provides a visual indication.

The ESP8266 continuously monitors the sensor readings. When excessive temperature or abnormal vibration is detected, the ESP8266 activates the buzzer, LED, and relay.



7. Algorithm:

1. Start the system.
2. Initialize the **ESP8266 NodeMCU, temperature sensor, vibration sensor, relay, buzzer, and LED.**
3. Read the **temperature value** from the temperature sensor.
4. Read the **vibration value** from the vibration sensor.
5. Compare the temperature and vibration values with the predefined safety limits.
6. Check whether **high temperature or abnormal vibration** is detected.
7. If an abnormal condition is detected, turn **ON the buzzer, LED, and relay.**
8. Display **“GENERATOR ALERT – MAINTENANCE REQUIRED”.**
9. Otherwise, keep the **buzzer, LED, and relay OFF** and display **“GENERATOR NORMAL”.**
10. Wait for a short interval and **repeat the monitoring process continuously.**

8. Coding:

```
#include <ESP8266WiFi.h>
```

```
#include <ThingSpeak.h>
```

```
#include <Wire.h>
```

```
#include <Adafruit_GFX.h>
```

```
#include <Adafruit_SSD1306.h>
```

```
#include <DHT.h>
```

```
// =====
```

```
// INDUSTRIAL DIESEL GENERATOR MONITORING SYSTEM
```

```
// ESP8266 + DHT11 + VOLTAGE SENSOR + OLED + BUZZER
```

```
// WiFi + ThingSpeak
```

```
// =====
```

```
// =====
```

```
// WIFI SETTINGS
```

```
// =====
```

```
const char* WIFI_SSID = "praveen";
```

```
const char* WIFI_PASSWORD = "praveen12345";
```

```
// =====
```

```
// THINGSPEAK
```

```
// =====
```

```
const char* API_KEY = "0IP4GOX0SMNK3JBV";
```

```
// Replace with your actual ThingSpeak Channel ID
```

```
unsigned long CHANNEL_ID = 1234567;
```

```
// =====
```

```
// DHT11
```

```
// =====
```

```
#define DHT_PIN 2
```

```
#define DHT_TYPE DHT11
```

```
DHT dht(DHT_PIN, DHT_TYPE);
```



```
// =====
```

```
// VOLTAGE SENSOR
```

```
// =====
```

```
#define VOLTAGE_PIN A0
```

```
// =====
```

```
// BUZZER
```

```
// =====
```

```
#define BUZZER_PIN 12
```

```
// D6 = GPIO12
```

```
// =====
```

```
// OLED
```

```
// =====
```

```
#define SCREEN_WIDTH 128
```

```
#define SCREEN_HEIGHT 64
```

```
#define OLED_RESET -1
```

```
#define OLED_ADDRESS 0x3C
```

```
Adafruit_SSD1306 display(  
  SCREEN_WIDTH,  
  SCREEN_HEIGHT,  
  &Wire,  
  OLED_RESET  
);
```

```
// =====  
// WIFI CLIENT  
// =====
```

```
WiFiClient client;
```

```
// =====  
// VOLTAGE CALIBRATION  
// =====
```

```
// For a common 0-25V voltage sensor module,  
// start with 5.0.  
//  
// This value MUST be calibrated using a multimeter.
```

```
float voltageCalibration = 5.0;
```

```
// =====
```

```
// ALARM LIMITS
```

```
// =====
```

```
float minimumVoltage = 200.0;
```

```
float maximumVoltage = 250.0;
```

```
float maximumTemperature = 50.0;
```

```
// =====
```

```
// THINGSPEAK UPDATE INTERVAL
```

```
// =====
```

```
unsigned long lastUpdate = 0;
```

```
const unsigned long UPDATE_INTERVAL = 20000;
```

```
// =====
```

```
// SETUP
```

```
// =====
```

```
void setup()
```

```
{
```

```
Serial.begin(9600);
```

```
delay(1000);
```

```
// =====
```

```
// I2C
```

```
// =====
```

```
// SDA = D2 / GPIO4
```

```
// SCL = D1 / GPIO5
```

```
Wire.begin(D2, D1);
```

```
// =====
```

```
// DHT11
```

```
// =====
```

```
dht.begin();
```

```
// =====
```

```
// BUZZER
```

```
// =====
```

```
pinMode(BUZZER_PIN, OUTPUT);
```

```
digitalWrite(  
    BUZZER_PIN,  
    LOW  
);
```

```
// =====  
// OLED  
// =====
```

```
if(  
    !display.begin(  
        SSD1306_SWITCHCAPVCC,  
        OLED_ADDRESS  
    )  
)  
{  
    Serial.println(  
        "OLED NOT FOUND!"  
    );  
}
```

```
while (true)  
{  
    delay(100);  
}  
}
```

```
// =====  
// OLED STARTUP SCREEN  
// =====
```

```
display.clearDisplay();
```

```
display.setTextColor(  
    SSD1306_WHITE  
);
```

```
display.setTextSize(1);
```

```
display.setCursor(0, 0);
```

```
display.println(  
    "DIESEL GENERATOR"  
);
```

```
display.setCursor(0, 15);
```

```
display.println(  
    "MONITORING SYSTEM"  
);
```

```
display.setCursor(0, 30);
```

```
display.println(  
  "Initializing..."  
);
```

```
display.display();
```

```
delay(2000);
```

```
// =====  
// WIFI  
// =====
```

```
Serial.println();
```

```
Serial.println(  
  "Connecting to WiFi..."  
);
```

```
WiFi.mode(WIFI_STA);
```

```
WiFi.begin(  
  WIFI_SSID,  
  WIFI_PASSWORD
```

```
);
```

```
int attempts = 0;
```

```
while (  
    WiFi.status() != WL_CONNECTED &&  
    attempts < 30  
)  
{  
    delay(500);  
  
    Serial.print(".");  
  
    attempts++;  
}
```

```
Serial.println();
```

```
if(  
    WiFi.status() == WL_CONNECTED  
)  
{  
    Serial.println(  

```



```
    "WiFi CONNECTED!"
);

Serial.print(
    "IP Address: "
);

Serial.println(
    WiFi.localIP()
);
}
else
{
    Serial.println(
        "WiFi CONNECTION FAILED!"
    );
}

// =====
// THINGSPEAK
// =====

ThingSpeak.begin(client);

Serial.println(
```

```
"ThingSpeak initialized."
);

Serial.println();

Serial.println(
  "SYSTEM READY"
);
}

// =====
// READ VOLTAGE
// =====

float readVoltage()
{
  int rawValue =
    analogRead(VOLTAGE_PIN);

  // ESP8266 ADC reading
  // 0 to 1023

  float adcVoltage =
    (rawValue / 1023.0) * 3.3;
```

```
float measuredVoltage =  
    adcVoltage *  
    voltageCalibration;
```

```
    return measuredVoltage;  
}
```

```
// =====  
// SHOW OLED  
// =====
```

```
void showOLED(  
    float voltage,  
    float temperature,  
    float humidity,  
    bool alarm  
)  
{  
    display.clearDisplay();  
  
    display.setTextSize(1);
```

```
display.setTextColor(  
    SSD1306_WHITE  
);
```

```
// -----  
// Voltage  
// -----
```

```
display.setCursor(0, 0);
```

```
display.print(  
    "Voltage: "  
);
```

```
display.print(  
    voltage,  
    1  
);
```

```
display.println(  
    " V"  
);
```

```
// -----  
// Temperature
```

```
// -----
```

```
display.setCursor(0, 15);
```

```
display.print(  
  "Temp: "  
);
```

```
display.print(  
  temperature,  
  1  
);
```

```
display.println(  
  " C"  
);
```

```
// -----
```

```
// Humidity
```

```
// -----
```

```
display.setCursor(0, 30);
```

```
display.print(  
  "Humidity: "  
);
```

```
display.print(  
    humidity,  
    1  
);
```

```
display.println(  
    "%"  
);
```

```
// -----  
// Status  
// -----
```

```
display.setCursor(0, 45);
```

```
if (alarm)  
{  
    display.println(  
        "STATUS: WARNING"  
    );  
}  
else  
{  
    display.println(  
        "STATUS: NORMAL"
```

```
);  
}
```

```
display.display();  
}
```

```
// =====  
// MAIN LOOP  
// =====
```

```
void loop()  
{  
  // =====  
  // READ DHT11  
  // =====
```

```
float temperature =  
  dht.readTemperature();
```

```
float humidity =  
  dht.readHumidity();
```

```
// =====
```

```
// READ VOLTAGE
```

```
// =====
```

```
float voltage =  
  readVoltage();
```

```
// =====
```

```
// CHECK DHT11
```

```
// =====
```

```
if(  
  isnan(temperature) ||  
  isnan(humidity)  
)  
{  
  Serial.println(  
    "DHT11 ERROR!"  
  );
```

```
  digitalWrite(  
    BUZZER_PIN,  
    HIGH  
  );
```

```
  delay(500);
```



```
digitalWrite(  
    BUZZER_PIN,  
    LOW  
);  
  
delay(2500);  
  
return;  
}  
  
// =====  
// CHECK ALARM CONDITIONS  
// =====  
  
bool alarm = false;  
  
if(  
    voltage < minimumVoltage  
)  
{  
    alarm = true;  
  
    Serial.println(  
        "WARNING: LOW VOLTAGE"  
    );
```

```
}
```

```
if(
```

```
    voltage > maximumVoltage
```

```
)
```

```
{
```

```
    alarm = true;
```

```
    Serial.println(
```

```
        "WARNING: HIGH VOLTAGE"
```

```
    );
```

```
}
```

```
if(
```

```
    temperature > maximumTemperature
```

```
)
```

```
{
```

```
    alarm = true;
```

```
    Serial.println(
```

```
        "WARNING: HIGH TEMPERATURE"
```

```
    );
```

```
}
```

```
// =====  
// BUZZER  
// =====
```

```
if (alarm)  
{  
  digitalWrite(  
    BUZZER_PIN,  
    HIGH  
  );  
}  
else  
{  
  digitalWrite(  
    BUZZER_PIN,  
    LOW  
  );  
}
```

```
// =====  
// SERIAL MONITOR  
// =====
```

```
Serial.println();
```

```
Serial.println(
```

```
"=====
```

```
);
```

```
Serial.print(
```

```
  "Generator Voltage : "
```

```
);
```

```
Serial.print(
```

```
  voltage,
```

```
  2
```

```
);
```

```
Serial.println(
```

```
  " V"
```

```
);
```

```
Serial.print(
```

```
  "Temperature   : "
```

```
);
```

```
Serial.print(
```

```
  temperature,
```

```
  2
```

```
);
```

```
Serial.println(  
  " C"  
);
```

```
Serial.print(  
  "Humidity      : "  
);
```

```
Serial.print(  
  humidity,  
  2  
);
```

```
Serial.println(  
  " %"  
);
```

```
if (alarm)  
{  
  Serial.println(  
    "Condition    : WARNING"  
  );
```

```
Serial.println(  
  "Buzzer        : ON"
```

```
);  
}  
else  
{  
  Serial.println(  
    "Condition      : NORMAL"  
  );  

```

```
  Serial.println(  
    "Buzzer        : OFF"  
  );  
}
```

```
// =====  
// OLED  
// =====
```

```
showOLED(  
  voltage,  
  temperature,  
  humidity,  
  alarm  
);
```

```
// =====
```

```
// THINGSPEAK
```

```
// =====
```

```
if(  
  WiFi.status() == WL_CONNECTED &&  
  millis() - lastUpdate >= UPDATE_INTERVAL  
)  
{  
  lastUpdate = millis();
```

```
  Serial.println();
```

```
  Serial.println(  
    "Sending data to ThingSpeak..."  
  );
```

```
  // Field 1 = Voltage
```

```
  ThingSpeak.setField(  
    1,  
    voltage  
  );
```

```
  // Field 2 = Temperature
```

```
ThingSpeak.setField(  
    2,  
    temperature  
);
```

```
// Field 3 = Humidity
```

```
ThingSpeak.setField(  
    3,  
    humidity  
);
```

```
// Field 4 = Alarm
```

```
int alarmStatus;
```

```
if (alarm)  
{  
    alarmStatus = 1;  
}  
else  
{  
    alarmStatus = 0;  
}
```



```
ThingSpeak.setField(  
    4,  
    alarmStatus  
);
```

```
int response =  
    ThingSpeak.writeFields(  
        CHANNEL_ID,  
        API_KEY  
    );
```

```
if (response == 200)  
{  
    Serial.println(  
        "ThingSpeak update SUCCESS!"  
    );  
}  
else  
{  
    Serial.print(  
        "ThingSpeak ERROR: "  
    );
```

```

Serial.println(
    response
);
}
}

```

```

Serial.println(
    "=====
");

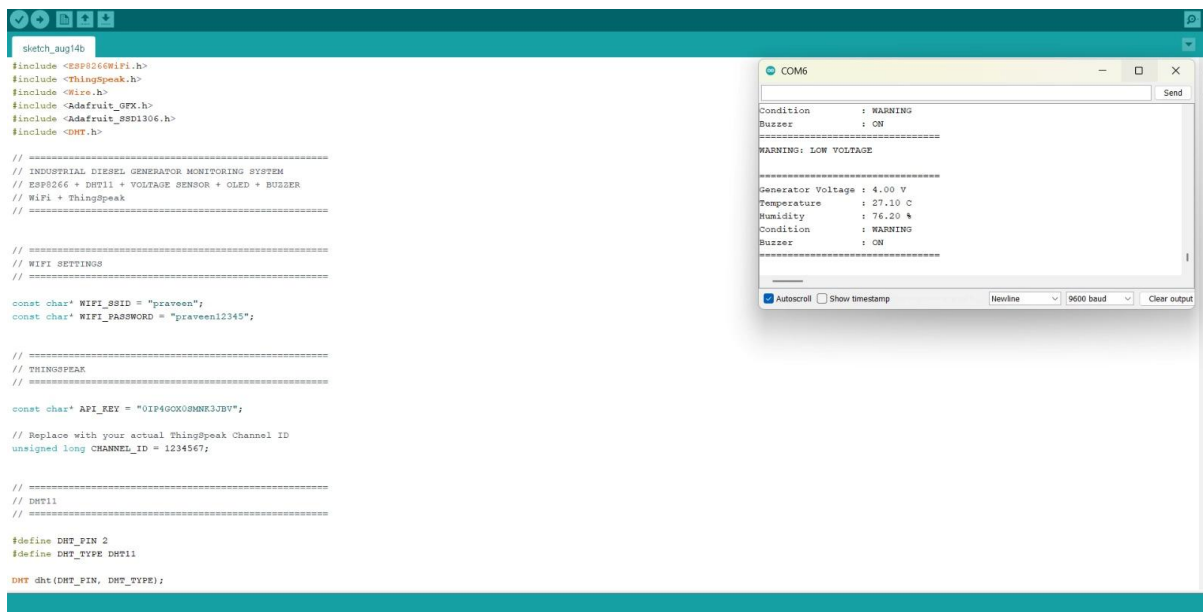
```

```

delay(3000);
}

```

Coding Output:



The screenshot shows the Arduino IDE interface. The sketch editor on the left contains the following code:

```

sketch_aug14b
#include <ESP8266WiFi.h>
#include <ThingSpeak.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>

// =====
// INDUSTRIAL DIESEL GENERATOR MONITORING SYSTEM
// ESP8266 + DHT11 + VOLTAGE SENSOR + OLED + BUZZER
// WiFi + ThingSpeak
// =====

// =====
// WIFI SETTINGS
// =====

const char* WIFI_SSID = "praveen";
const char* WIFI_PASSWORD = "praveen12345";

// =====
// THINGSPEAK
// =====

const char* API_KEY = "01P4G0X0SMNH3JBV";

// Replace with your actual ThingSpeak Channel ID
unsigned long CHANNEL_ID = 1234567;

// =====
// DHT11
// =====

#define DHT_PIN 2
#define DHT_TYPE DHT11

DHT dht(DHT_PIN, DHT_TYPE);

```

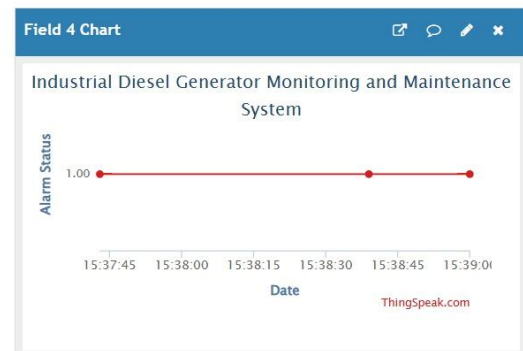
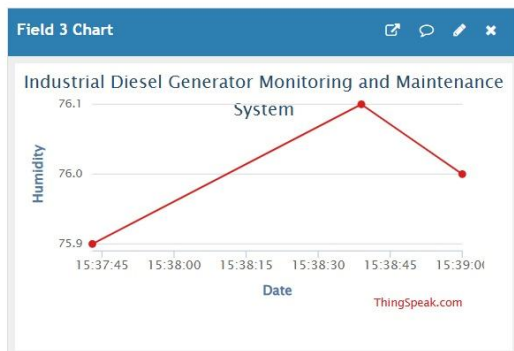
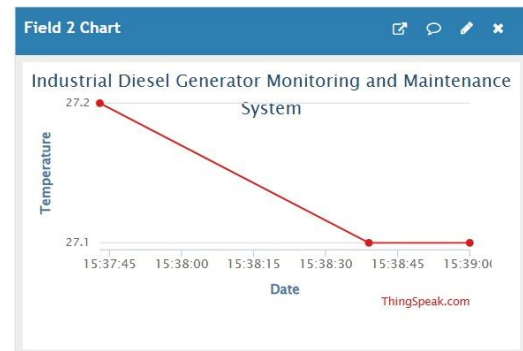
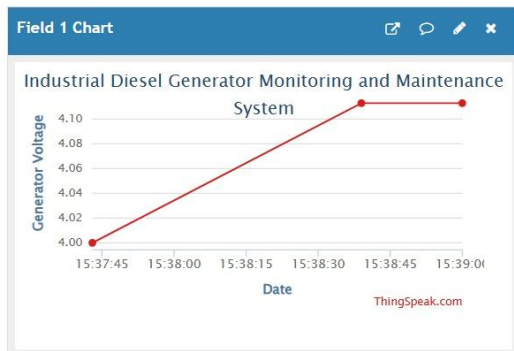
The serial monitor on the right, titled 'COM6', shows the output of the sketch:

```

Condition      : WARNING
Buzzzer       : ON
=====
WARNING: LOW VOLTAGE
=====
Generator Voltage : 4.00 V
Temperature      : 27.10 C
Humidity         : 76.20 %
Condition       : WARNING
Buzzzer        : ON
=====

```

The serial monitor settings are: Baud rate: 9600, Data bits: 8, Parity: None, Stop bits: 1. The 'Autoscroll' checkbox is checked.



9. System Working:

Power ON the System:

The ESP8266 NodeMCU is powered ON.

Initialize Components:

The temperature sensor, vibration sensor, relay module, buzzer, and LED are initialized.

Monitor Generator Temperature

The temperature sensor continuously monitors the generator operating temperature.

Monitor Generator Vibration

The vibration sensor continuously detects generator vibration.

Process Sensor Data

The ESP8266 receives and processes the sensor readings.

Check Generator Condition

- The temperature and vibration values are compared with predefined limits.

Abnormal Condition

If excessive temperature or abnormal vibration is detected:

- Buzzer → ON
- LED → ON
- Relay → ON
- Generator Alert → Activated
- Maintenance → Required

Normal Condition

the generator operates within normal limits:

- Buzzer → OFF
- LED → OFF
- Relay → OFF
- Generator Status → Normal

Repeat the Process:

- The system continuously monitors the generator condition and checks for abnormal operating parameters.

10. Prototype Implementation:

The prototype is implemented using an ESP8266 NodeMCU, temperature sensor, vibration sensor, relay module, buzzer, LED, breadboard, and jumper wires.

The temperature sensor monitors the operating temperature of the generator, while the vibration sensor detects abnormal vibration. The sensor readings are processed by the ESP8266.

Hardware Setup

Connect the Temperature Sensor output → A0 of ESP8266.

Connect the Vibration Sensor OUT pin → D4 (GPIO2).

Connect the Relay IN pin → D5 (GPIO14).

Connect the Buzzer → D6 (GPIO12).

Connect the LED → D7 (GPIO13) through a suitable resistor.

Connect the sensor power pins to a suitable power supply.

Connect all GND terminals to a common ground.

Upload the program to the ESP8266 using the Arduino IDE.

Power ON the prototype.

Observe the temperature and vibration sensor readings.

Create an abnormal condition by increasing the sensor value or vibration.

Observe the activation of the buzzer, LED, and relay.

The system continuously monitors the generator condition.

Prototype Working:

- The ESP8266 NodeMCU is powered ON.
- The temperature sensor monitors generator temperature.
- The vibration sensor detects abnormal generator vibration.
- The sensor readings are sent to the ESP8266.
- The ESP8266 compares the readings with predefined limits.
- If an abnormal condition is detected, the buzzer turns ON.
- The LED provides a visual warning.
- The relay is activated for generator protection or shutdown.
- A maintenance alert is displayed.
- If the generator condition is normal, all warning devices remain OFF.
- The monitoring process continues automatically.

11. Bill of Materials:

S.No.	Component	Specification	Quantity
1	ESP8266	NodeMCU Wi-Fi Development Board	1
2	Temperature Sensor	Analog Temperature Sensor	1
3	Vibration Sensor	Vibration Detection Module	1
4	Relay Module	1-Channel Relay	1
5	Buzzer	3.3V/5V Buzzer	1
6	LED	Standard Indicator LED	1
7	Resistor	Suitable LED Current Limiting Resistor	1
8	Breadboard	Standard Prototype Board	1
9	Jumper Wires	Male-Male / Male-Female	1 Set
10	USB Cable	Micro-USB	1
11	Power Supply	Suitable Regulated Supply	1

12. Results & Performance Analysis:

- The temperature sensor successfully monitors the generator operating temperature.
- The vibration sensor successfully detects abnormal generator vibration.
- The ESP8266 NodeMCU successfully receives and processes the sensor readings.
- The system provides continuous generator condition monitoring.
- When high temperature or abnormal vibration is detected, the buzzer is activated.
- The LED provides a visual warning indication.
- The relay can be activated for generator protection or shutdown.
- When the generator condition returns to normal, the warning devices return to their normal state.

Results

The temperature sensor successfully monitors generator temperature.

The vibration sensor successfully detects abnormal vibration.

The ESP8266 successfully processes the sensor readings.

The system continuously monitors generator health.

Abnormal operating conditions are detected automatically.

The buzzer provides an audible maintenance alert.

The LED provides a visual warning.

The relay demonstrates generator protection or shutdown.

The prototype demonstrates the basic operation of an **Industrial Diesel Generator Monitoring and Maintenance System**.

Performance Analysis:

Parameter	Expected Performance
Temperature Monitoring	Continuous
Vibration Monitoring	Continuous
Sensor Response	Real-time
Abnormal Condition Detection	Automatic
Maintenance Alert	Buzzer ON
Visual Warning	LED ON
Generator Protection	Relay ON
Normal Condition	Buzzer and LED OFF

